Traffic Light Controller

Release 1.0.0

HDL Team

Mar 11, 2026

Contents

1 test123 — Overview **1**
1.1 Modules . . . 1

2 top **3**
2.1 top — Entity . . . 3
2.1.1 Annotated Source . . . 3
2.2 top — State Machine . . . 5
2.3 top — Timing Diagrams . . . 6
2.4 top — Clock Domain Analysis . . . 6
2.5 Registers . . . 6
2.6 Submodules . . . 6
2.6.1 blinky . . . 6
2.6.1.1 blinky — Entity . . . 6
2.6.1.2 blinky State Machine . . . 9
2.6.1.3 blinky — Timing Diagrams . . . 10
2.6.1.4 Processes . . . 11
2.6.1.5 blinky — Clock Domain Analysis . . . 14
2.6.2 cfg_sync . . . 14
2.6.2.1 cfg_sync — Entity . . . 14
2.6.2.2 cfg_sync — State Machine . . . 17
2.6.2.3 cfg_sync — Timing Diagrams . . . 17
2.6.2.4 Processes . . . 18
2.6.2.5 cfg_sync — Clock Domain Analysis . . . 20
2.6.3 pwm_controller . . . 21
2.6.3.1 pwm_controller — Entity . . . 21
2.6.3.2 pwm_controller State Machine . . . 23
2.6.3.3 pwm_controller — Timing Diagrams . . . 24
2.6.3.4 Processes . . . 25
2.6.3.5 pwm_controller — Clock Domain Analysis . . . 27
2.6.4 traffic_light . . . 28
2.6.4.1 traffic_light — Entity . . . 28
2.6.4.2 traffic_light State Machine . . . 32
2.6.4.3 traffic_light — Timing Diagrams . . . 32
2.6.4.4 Processes . . . 33
2.6.4.5 traffic_light — Clock Domain Analysis . . . 36

3 Design Hierarchy **37**
3.1 Instantiation Tree . . . 37
3.2 Module List . . . 37

4 Indices **39**

Chapter 1

test123 — Overview

This project contains 5 VHDL/SV modules.

Top-level module: *top*

1.1 Modules

Module	Source File	Description
traffic_light	`traffic_light.vhd`	Traffic light controller. Implements a standard UK traffic light sequencer with four states:
pwm_controlle	`pwm_controller.sv`	PWM controller module. Generates a variable duty-cycle PWM signal using a 3-state FSM:
cfg_sync	`cfg_sync.vhd`	cfg_sync entity.
top	`top.vhd`	top entity.
blinky	`blinky.vhd`	Blinky LED controller. Implements a 3-state FSM that drives an LED through a repeating

Chapter 2

top

2.1 top — Entity

Source file: `src/top.vhd`.

ENTITY **`top`** IS

`Generics`

Generic	Type	Default	Description

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	System clock 50 MHz.
rst	std_logic	in	Synchronous active-high reset (sys_clk domain).
cfg_clk	std_logic	in	Config bus clock (slow, e.g. 10 MHz from external MCU).
cfg_rst	std_logic	in	Synchronous active-high reset (cfg_clk domain).
mode_sel	std_logic	in	Config inputs (cfg_clk domain).
duty_cycle	std_logic_vector(7 downto 0)	in	
red_out	std_logic	out	Traffic light outputs.
amber_out	std_logic	out	
green_out	std_logic	out	
led_on	std_logic	out	Blinky LED outputs.
led_fade	std_logic	out	
led_off	std_logic	out	
pwm_out	std_logic	out	PWM output for LED fading (shared).

2.1.1 Annotated Source

Listing 1: src/top.vhd

```
-- Top-level integration module.
--
-- Instantiates the traffic_light controller, blinky LED module, PWM
-- controller, and cfg_sync. Runtime configuration (blinky enable and PWM
-- duty cycle) arrives on a slow config bus clock and is synchronised into
-- the main 50 MHz domain by cfg_sync.
library ieee;
use ieee.std_logic_1164.all;

entity top is
    port (
        -- System clock 50 MHz.
        clk          : in  std_logic;
        -- Synchronous active-high reset (sys_clk domain).
        rst          : in  std_logic;
        -- Config bus clock (slow, e.g. 10 MHz from external MCU).
        cfg_clk      : in  std_logic;
        -- Synchronous active-high reset (cfg_clk domain).
        cfg_rst      : in  std_logic;
        -- Config inputs (cfg_clk domain).
        mode_sel     : in  std_logic;
        duty_cycle   : in  std_logic_vector(7 downto 0);
        -- Traffic light outputs.
        red_out      : out std_logic;
        amber_out    : out std_logic;
        green_out    : out std_logic;
        -- Blinky LED outputs.
        led_on       : out std_logic;
        led_fade     : out std_logic;
        led_off      : out std_logic;
        -- PWM output for LED fading (shared).
        pwm_out      : out std_logic
    );
end entity top;

architecture rtl of top is

    signal timer_exp   : std_logic;
    signal blinky_en   : std_logic;
    signal pwm_duty    : std_logic_vector(7 downto 0);
    signal period_done : std_logic;

begin

    -- u_cfg_sync: Synchronise config bus inputs into the system clock domain.
    u_cfg_sync : entity work.cfg_sync
        port map (
            sys_clk    => clk,
            sys_rst    => rst,
            cfg_clk    => cfg_clk,
            cfg_rst    => cfg_rst,
```

(continues on next page)

(continued from previous page)

```
            mode_sel   => mode_sel,
            duty_cycle => duty_cycle,
            blinky_en  => blinky_en,
            pwm_duty   => pwm_duty
        );

    -- u_traffic_light: Traffic light FSM instance.
    u_traffic_light : entity work.traffic_light
        port map (
            clk       => clk,
            rst       => rst,
            timer_exp => timer_exp,
            red_out   => red_out,
            amber_out => amber_out,
            green_out => green_out
        );

    -- u_blinky: Blinky LED FSM instance.
    u_blinky : entity work.blinky
        port map (
            clk       => clk,
            rst       => rst,
            en        => blinky_en,
            timer_exp => timer_exp,
            led_on    => led_on,
            led_fade  => led_fade,
            led_off   => led_off
        );

    -- u_pwm: PWM controller shared between traffic light and blinky fade.
    u_pwm : entity work.pwm_controller
        port map (
            clk         => clk,
            rst         => rst,
            en          => blinky_en,
            duty_i      => pwm_duty,
            pwm_o       => pwm_out,
            period_done => period_done
        );

end architecture rtl;
```

2.2 top — State Machine

No FSM detected in `top.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.3 top — Timing Diagrams

All timing diagrams extracted from `top.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.4 top — Clock Domain Analysis

Auto-extracted from `top.vhd`.

> **Note**
>
> No clocked processes detected. This module appears to be purely combinational.

2.5 Registers

2.6 Submodules

2.6.1 blinky

2.6.1.1 blinky — Entity

Source file: `src/blinky.vhd`.

ENTITY **`blinky`** IS

Blinky LED controller.

Implements a 3-state FSM that drives an LED through a repeating ON -> FADE -> OFF sequence. Each state is held for a configurable number of clock cycles determined by an external timer pulse.

State sequence: ON -> FADE -> OFF -> ON -> … **`Generics`**

Generic	Type	Default	Description

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	System clock, rising-edge triggered.
rst	std_logic	in	Synchronous active-high reset. Returns FSM to OFF state.
en	std_logic	in	Enable signal. FSM only advances when en is asserted.
timer_exp	std_logic	in	Phase timer expiry pulse. Must be high for exactly one cycle.
led_on	std_logic	out	LED full-brightness drive.
led_fade	std_logic	out	LED half-brightness PWM drive (fade effect).
led_off	std_logic	out	LED off indicator (all drives deasserted).

Annotated Source

Listing 2: src/blinky.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Blinky LED controller.
--
-- Implements a 3-state FSM that drives an LED through a repeating
-- ON -> FADE -> OFF sequence. Each state is held for a configurable
-- number of clock cycles determined by an external timer pulse.
--
-- State sequence: ON -> FADE -> OFF -> ON -> ...
entity blinky is
    port (
        -- System clock, rising-edge triggered.
        clk       : in  std_logic;
        -- Synchronous active-high reset. Returns FSM to OFF state.
        rst       : in  std_logic;
        -- Enable signal. FSM only advances when en is asserted.
        en        : in  std_logic;
        -- Phase timer expiry pulse. Must be high for exactly one cycle.
        timer_exp : in  std_logic;
        -- LED full-brightness drive.
        led_on    : out std_logic;
        -- LED half-brightness PWM drive (fade effect).
        led_fade  : out std_logic;
        -- LED off indicator (all drives deasserted).
        led_off   : out std_logic
    );
end entity blinky;

-- RTL implementation using a 3-process Moore FSM style.
architecture rtl of blinky is

    -- FSM state type encoding the three LED phases.
    type t_state is (
        -- LED fully illuminated.
        ON_STATE,
        -- LED dimming via PWM (fade transition).
        FADE_STATE,
        -- LED completely off.
        OFF_STATE
    );

    -- Current state register, initialised to OFF at power-on.
    signal state      : t_state := OFF_STATE;
    -- Next state combinational signal.
    signal next_state : t_state;

begin
```

(continues on next page)

(continued from previous page)

```

    -- p_state_reg: Clocked state register.
    --
    -- Captures next_state on each rising edge of clk.
    -- Synchronous reset drives state back to OFF_STATE.
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",       "wave": "P....." },
    --      { "name": "rst",       "wave": "1.0....." },
    --      { "name": "en",        "wave": "0....." },
    --      { "name": "state",     "wave": "=.=....", "data": ["(any)","OFF"] },
    --      { "name": "led_on",    "wave": "x.0....." },
    --      { "name": "led_fade",  "wave": "x.0....." },
    --      { "name": "led_off",   "wave": "x.1....." }
    --    ]}
    p_state_reg : process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state <= OFF_STATE;
            else
                state <= next_state;
            end if;
        end if;
    end process p_state_reg;

    -- p_next_state: Combinational next-state logic.
    --
    -- FSM only advances when both en and timer_exp are asserted.
    -- If en is deasserted the state holds regardless of timer_exp.
    --
    -- Transitions:
    --   OFF_STATE  + en + timer_exp -> ON_STATE
    --   ON_STATE   + en + timer_exp -> FADE_STATE
    --   FADE_STATE + en + timer_exp -> OFF_STATE
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",       "wave": "P....." },
    --      { "name": "en",        "wave": "01.....0" },
    --      { "name": "timer_exp", "wave": "0..1..1..1..0." },
    --      { "name": "state",     "wave": "=..=..=..=..=.",
    --        "data": ["OFF","ON","FADE","OFF","OFF"] },
    --      { "name": "led_on",    "wave": "0..1..0..0..0." },
    --      { "name": "led_fade",  "wave": "0..0..1..0..0." },
    --      { "name": "led_off",   "wave": "1..0..0..1..1." }
    --    ]}
    p_next_state : process(state, en, timer_exp)
    begin
```

(continues on next page)

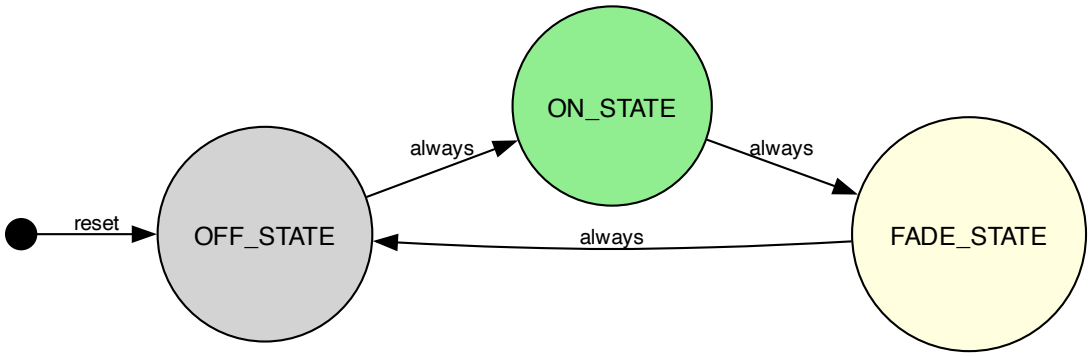
(continued from previous page)

```
        next_state <= state;

        if en = '1' and timer_exp = '1' then
            case state is
                when OFF_STATE  => next_state <= ON_STATE;
                when ON_STATE   => next_state <= FADE_STATE;
                when FADE_STATE => next_state <= OFF_STATE;
            end case;
        end if;
    end process p_next_state;

    -- p_outputs: Combinational Moore output decode.
    --
    -- Drives LED outputs based solely on current state.
    -- All outputs default to 0 before the case statement to
    -- prevent latches during synthesis.
    --
    -- Output truth table:
    --   OFF_STATE  -> led_on=0 led_fade=0 led_off=1
    --   ON_STATE   -> led_on=1 led_fade=0 led_off=0
    --   FADE_STATE -> led_on=0 led_fade=1 led_off=0
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "state",    "wave": "=.=.=.=.",
    --        "data": ["OFF","ON","FADE","OFF"] },
    --      { "name": "led_on",   "wave": "0.1.0.0." },
    --      { "name": "led_fade", "wave": "0.0.1.0." },
    --      { "name": "led_off",  "wave": "1.0.0.1." }
    --    ]}
    p_outputs : process(state)
    begin
        led_on   <= '0';
        led_fade <= '0';
        led_off  <= '0';

        case state is
            when OFF_STATE  => led_off  <= '1';
            when ON_STATE   => led_on   <= '1';
            when FADE_STATE => led_fade <= '1';
        end case;
    end process p_outputs;

end architecture rtl;
```

2.6.1.2 blinky State Machine

Auto-extracted from `blinky.vhd`.

State Transition Diagram



State Output Table

State	led_on	led_fade	led_off
OFF_STATE	0	0	1
ON_STATE	1	0	0
FADE_STATE	0	1	0

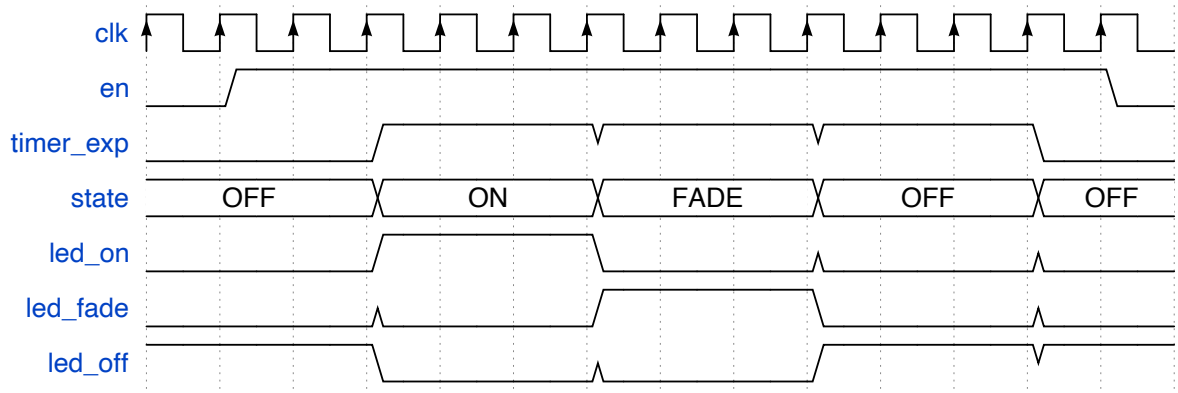
Transitions

From	To	Condition
OFF_STATE	ON_STATE	(always)
ON_STATE	FADE_STATE	(always)
FADE_STATE	OFF_STATE	(always)

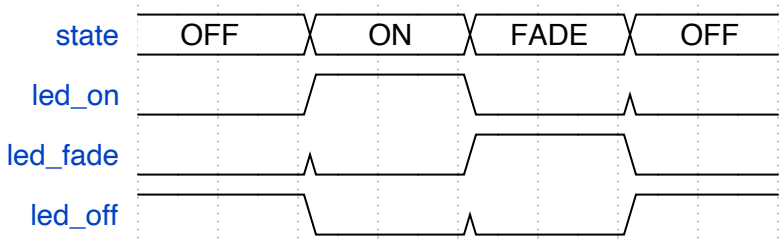
2.6.1.3 blinky — Timing Diagrams

All timing diagrams extracted from `blinky.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

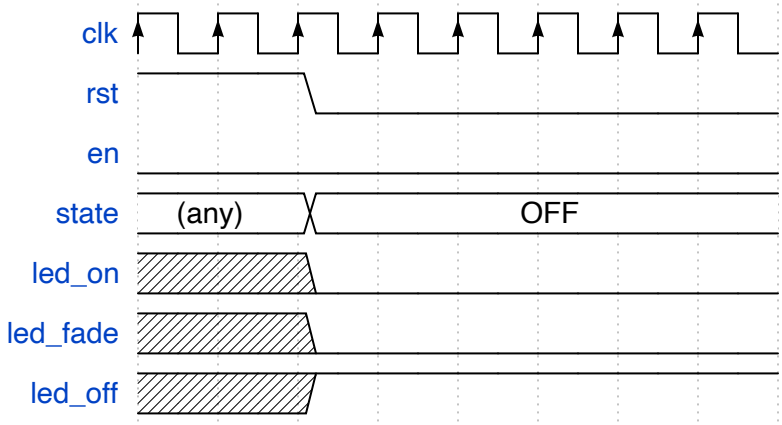
P Next State



P Outputs



P State Reg



2.6.1.4 Processes

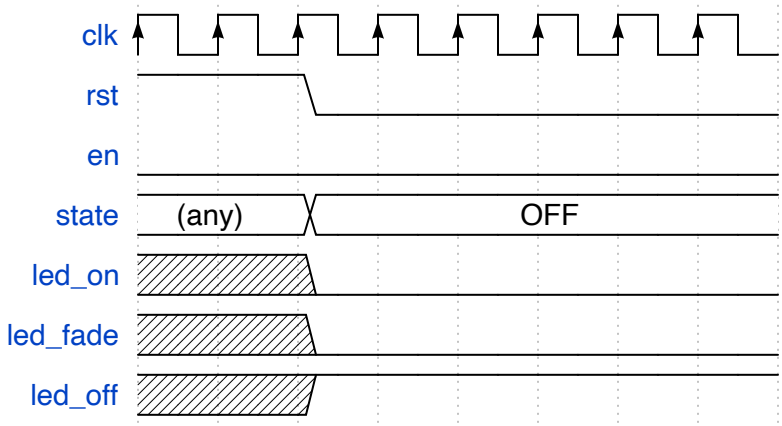
Auto-extracted from `blinky.vhd`.

p_state_reg

Source file: `blinky.vhd`
Source line: 67
Block type: `process`
Sensitivity list: `clk`

Description

p_state_reg: Clocked state register. Captures next_state on each rising edge of clk. Synchronous reset drives state back to OFF_STATE.



Source

```
p_state_reg : process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            state <= OFF_STATE;
        else
            state <= next_state;
        end if;
    end if;
end process p_state_reg;
```

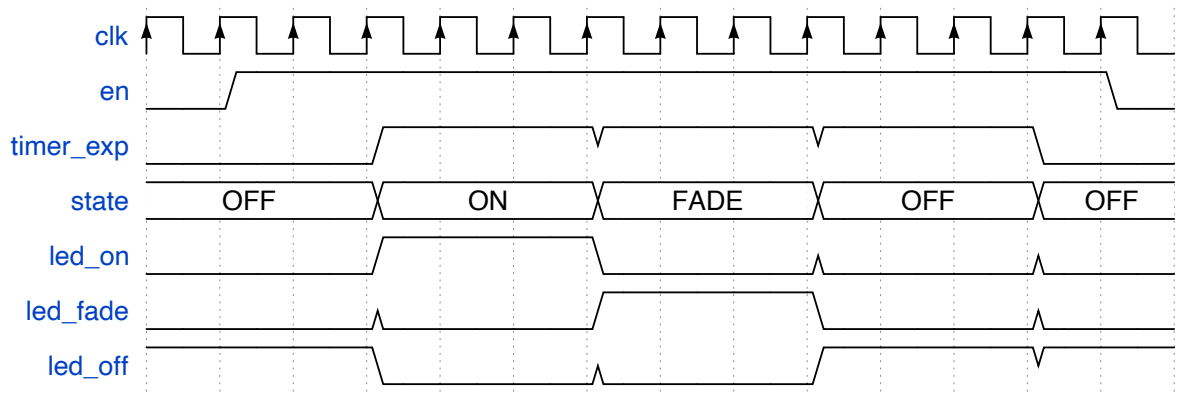
p_next_state

Source file: `blinky.vhd`
Source line: 100
Block type: `process`
Sensitivity list: `state`, `en`, `timer_exp`

Description

p_next_state: Combinational next-state logic. FSM only advances when both en and timer_exp are asserted. If en is deasserted the state holds regardless of timer_exp. Transitions: .. code-block:: none

> OFF_STATE + en + timer_exp -> ON_STATE ON_STATE + en + timer_exp -> FADE_STATE FADE_STATE + en + timer_exp -> OFF_STATE



Source

```
p_next_state : process(state, en, timer_exp)
begin
    next_state <= state;

    if en = '1' and timer_exp = '1' then
        case state is
            when OFF_STATE  => next_state <= ON_STATE;
            when ON_STATE   => next_state <= FADE_STATE;
            when FADE_STATE => next_state <= OFF_STATE;
        end case;
    end if;
end process p_next_state;
```

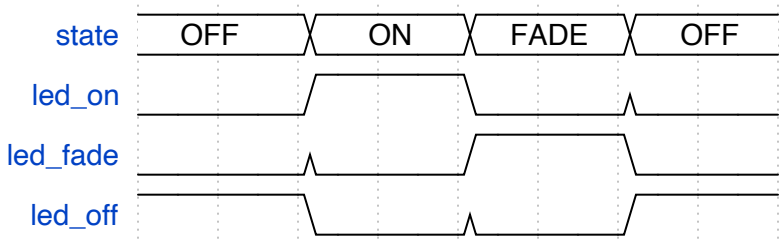
p_outputs

Source file: `blinky.vhd`
Source line: 133
Block type: `process`
Sensitivity list: `state`

Description

p_outputs: Combinational Moore output decode. Drives LED outputs based solely on current state. All outputs default to 0 before the case statement to prevent latches during synthesis. Output truth table: .. code-block:: none

OFF_STATE -> led_on=0 led_fade=0 led_off=1 ON_STATE -> led_on=1 led_fade=0 led_off=0
FADE_STATE -> led_on=0 led_fade=1 led_off=0



Source

```
p_outputs : process(state)
begin
    led_on   <= '0';
    led_fade <= '0';
    led_off  <= '0';

    case state is
        when OFF_STATE  => led_off  <= '1';
        when ON_STATE   => led_on   <= '1';
        when FADE_STATE => led_fade <= '1';
    end case;
end process p_outputs;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_state_reg	process	clk	67	✓
p_next_state	process	state, en, timer_exp	100	✓
p_outputs	process	state	133	✓

2.6.1.5 blinky — Clock Domain Analysis

Auto-extracted from `blinky.vhd`.

> **Note**
>
> No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
clk	p_state_reg

2.6.2 cfg_sync

2.6.2.1 cfg_sync — Entity

Source file: `src/cfg_sync.vhd`.

ENTITY **cfg_sync** IS

Generics

Generic	Type	Default	Description

Ports

Port	Type	Mode	Description
sys_clk	std_logic	in	System clock domain (50 MHz).
sys_rst	std_logic	in	Synchronous active-high reset (sys_clk domain).
cfg_clk	std_logic	in	Config clock domain (slow bus clock).
cfg_rst	std_logic	in	Synchronous active-high reset (cfg_clk domain).
mode_sel	std_logic	in	Config inputs (cfg_clk domain). Blinky enable: 1 = blinky LED active.
duty_cycle	std_logic_vector(7 downto 0)	in	PWM duty cycle: 0x00 = 0 %, 0xFF = 100 %.
blinky_en	std_logic	out	Outputs (sys_clk domain). Synchronised blinky enable (two-flop).
pwm_duty	std_logic_vector(7 downto 0)	out	PWM duty cycle (unsynchronised crossing).

Annotated Source

Listing 3: src/cfg_sync.vhd

```vhdl
-- Configuration synchroniser.
--
-- Receives runtime configuration from a slow config bus clock domain
-- (cfg_clk, e.g. 10 MHz from an external microcontroller) and makes it
-- available in the main system clock domain (sys_clk, 50 MHz).
--
-- mode_sel   crosses via a two-flop synchroniser (safe).
-- duty_cycle crosses as a raw multi-bit bus with no synchronisation
--            (unsafe - included to demonstrate CDC detection; a real
--            design should use a handshake or grey-code scheme here).
library ieee;
use ieee.std_logic_1164.all;

entity cfg_sync is
    port (
        -- System clock domain (50 MHz).
        sys_clk    : in  std_logic;
        -- Synchronous active-high reset (sys_clk domain).
        sys_rst    : in  std_logic;
        -- Config clock domain (slow bus clock).
        cfg_clk    : in  std_logic;
        -- Synchronous active-high reset (cfg_clk domain).
        cfg_rst    : in  std_logic;
        -- Config inputs (cfg_clk domain).
        -- Blinky enable: 1 = blinky LED active.
        mode_sel   : in  std_logic;
        -- PWM duty cycle: 0x00 = 0 %, 0xFF = 100 %.
        duty_cycle : in  std_logic_vector(7 downto 0);
        -- Outputs (sys_clk domain).
        -- Synchronised blinky enable (two-flop).
        blinky_en  : out std_logic;
        -- PWM duty cycle (unsynchronised crossing).
        pwm_duty   : out std_logic_vector(7 downto 0)
```

(continues on next page)

(continued from previous page)

```
    );
end entity cfg_sync;

architecture rtl of cfg_sync is

    -- cfg_clk domain registers
    signal mode_sel_r   : std_logic;
    signal duty_cycle_r : std_logic_vector(7 downto 0);

    -- sys_clk domain: two-flop synchroniser pipeline for mode_sel_r
    signal mode_meta    : std_logic;
    signal mode_sync_r  : std_logic;

begin

    -- p_cfg_reg: Register config inputs in the cfg_clk domain.
    --
    -- Provides a stable registered version of mode_sel and duty_cycle
    -- before they cross into the sys_clk domain.
    p_cfg_reg : process(cfg_clk) is
    begin
        if rising_edge(cfg_clk) then
            if cfg_rst = '1' then
                mode_sel_r   <= '0';
                duty_cycle_r <= (others => '0');
            else
                mode_sel_r   <= mode_sel;
                duty_cycle_r <= duty_cycle;
            end if;
        end if;
    end process p_cfg_reg;

    -- p_mode_sync: Two-flop synchroniser for mode_sel_r (cfg_clk → sys_clk).
    --
    -- mode_sel_r is sampled twice in the sys_clk domain to resolve
    -- metastability before the signal is used by downstream logic.
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "cfg_clk",    "wave": "p........" },
    --      { "name": "sys_clk",    "wave": "P........" },
    --      { "name": "mode_sel_r", "wave": "0...1...." },
    --      { "name": "mode_meta",  "wave": "0....1..." },
    --      { "name": "mode_sync_r","wave": "0.....1.." }
    --    ]}
    p_mode_sync : process(sys_clk) is
    begin
        if rising_edge(sys_clk) then
            if sys_rst = '1' then
                mode_meta   <= '0';
                mode_sync_r <= '0';
```

(continues on next page)

(continued from previous page)

```
            else
                mode_meta   <= mode_sel_r;
                mode_sync_r <= mode_meta;
            end if;
        end if;
    end process p_mode_sync;

    -- p_duty_capture: Capture duty_cycle_r directly in the sys_clk domain.
    --
    -- WARNING: This is an unsynchronised multi-bit crossing. duty_cycle_r is
    -- driven in the cfg_clk domain and sampled here without a handshake or
    -- grey-code scheme. Glitches are possible if both bits change together
    -- near a sys_clk edge. A proper implementation should use a handshake
    -- protocol or an async FIFO.
    p_duty_capture : process(sys_clk) is
    begin
        if rising_edge(sys_clk) then
            if sys_rst = '1' then
                pwm_duty <= (others => '0');
            else
                pwm_duty <= duty_cycle_r;
            end if;
        end if;
    end process p_duty_capture;

    blinky_en <= mode_sync_r;

end architecture rtl;
```

2.6.2.2 cfg_sync — State Machine

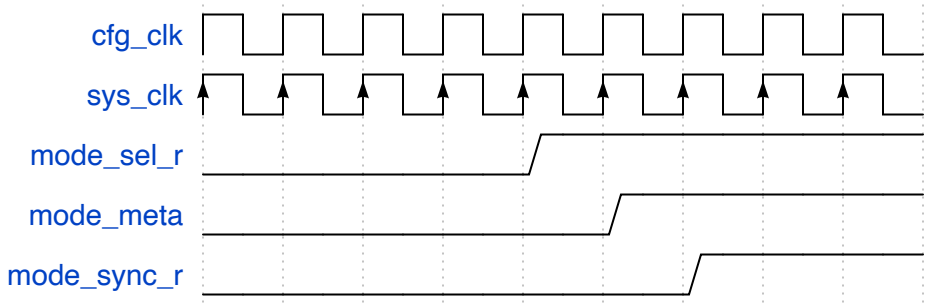
No FSM detected in `cfg_sync.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.6.2.3 cfg_sync — Timing Diagrams

All timing diagrams extracted from `cfg_sync.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

P Mode Sync



2.6.2.4 Processes

Auto-extracted from `cfg_sync.vhd`.

p_cfg_reg

Source file: `cfg_sync.vhd`
Source line: 53
Block type: `process`
Sensitivity list: `cfg_clk`

Description

p_cfg_reg: Register config inputs in the cfg_clk domain. Provides a stable registered version of mode_sel and duty_cycle before they cross into the sys_clk domain.

Source

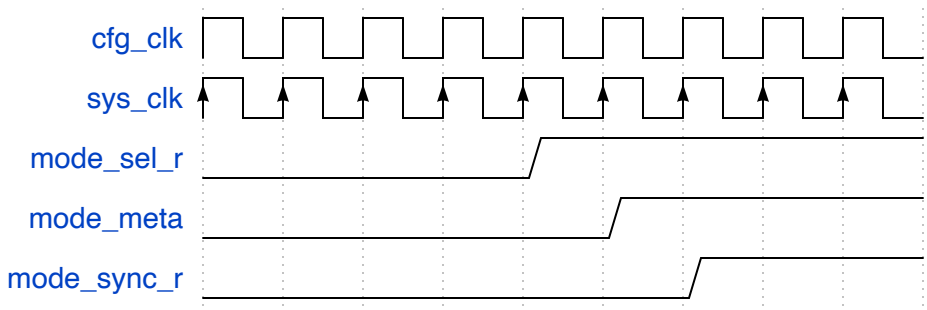
```
p_cfg_reg : process(cfg_clk) is
begin
    if rising_edge(cfg_clk) then
        if cfg_rst = '1' then
            mode_sel_r   <= '0';
            duty_cycle_r <= (others => '0');
        else
            mode_sel_r   <= mode_sel;
            duty_cycle_r <= duty_cycle;
        end if;
    end if;
end process p_cfg_reg;
```

p_mode_sync

Source file: `cfg_sync.vhd`
Source line: 80
Block type: `process`
Sensitivity list: `sys_clk`

Description

p_mode_sync: Two-flop synchroniser for mode_sel_r (cfg_clk → sys_clk). mode_sel_r is sampled twice in the sys_clk domain to resolve metastability before the signal is used by downstream logic.



Source

```
p_mode_sync : process(sys_clk) is
begin
    if rising_edge(sys_clk) then
        if sys_rst = '1' then
            mode_meta   <= '0';
            mode_sync_r <= '0';
        else
            mode_meta   <= mode_sel_r;
            mode_sync_r <= mode_meta;
        end if;
    end if;
end process p_mode_sync;
```

p_duty_capture

Source file: `cfg_sync.vhd`
Source line: 100
Block type: `process`
Sensitivity list: `sys_clk`

Description

p_duty_capture: Capture duty_cycle_r directly in the sys_clk domain. WARNING: This is an unsynchronised multi-bit crossing. duty_cycle_r is driven in the cfg_clk domain and sampled here without a handshake or grey-code scheme. Glitches are possible if both bits change together near a sys_clk edge. A proper implementation should use a handshake protocol or an async FIFO.

Source

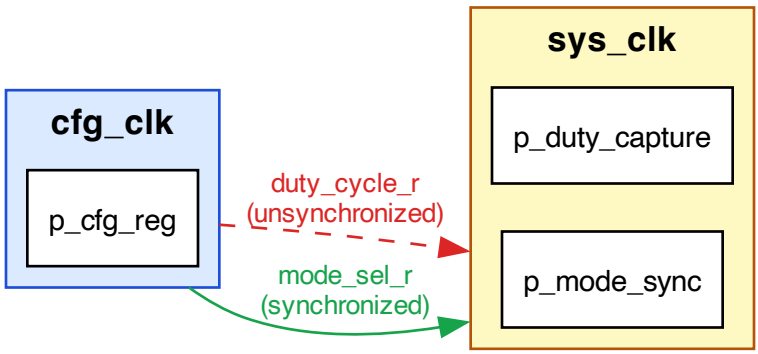
```
p_duty_capture : process(sys_clk) is
begin
    if rising_edge(sys_clk) then
        if sys_rst = '1' then
            pwm_duty <= (others => '0');
        else
            pwm_duty <= duty_cycle_r;
        end if;
    end if;
end process p_duty_capture;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_cfg_reg	`process`	`cfg_clk`	53	—
p_mode_sync	`process`	`sys_clk`	80	✓
p_duty_capture	`process`	`sys_clk`	100	—

2.6.2.5 cfg_sync — Clock Domain Analysis

Auto-extracted from `cfg_sync.vhd`.



Clock Domains

Clock	Clocked Processes
`cfg_clk`	`p_cfg_reg`
`sys_clk`	`p_mode_sync`, `p_duty_capture`

Warning

The following signal(s) cross clock domains without a detected synchronizer: `duty_cycle_r`.

Signal Crossings

Signal	Source Domain	Destination Domain	Synchronized
`duty_cycle_r`	`cfg_clk`	`sys_clk`	**No**
`mode_sel_r`	`cfg_clk`	`sys_clk`	Yes *(two-flop)*

2.6.3 pwm_controller

2.6.3.1 pwm_controller — Entity

Source file: `src/pwm_controller.sv`.

Annotated Source

Listing 4: src/pwm_controller.sv

```
// PWM controller module.
//
// Generates a variable duty-cycle PWM signal using a 3-state FSM:
// IDLE -> COUNTING -> DONE. The duty cycle is set via the duty_i port.
// The period is fixed at 256 clock cycles.
module pwm_controller #(
    parameter int WIDTH = 8
)(
    // System clock, rising-edge triggered.
    input  logic              clk,
    // Synchronous active-high reset.
    input  logic              rst,
    // Enable signal. FSM runs when asserted.
    input  logic              en,
    // Duty cycle: 0=0%, 255=100%.
    input  logic [WIDTH-1:0]  duty_i,
    // PWM output signal.
    output logic              pwm_o,
    // Active high when one full PWM period has completed.
    output logic              period_done
);

    // FSM state encoding
    typedef enum logic [1:0] {
        // Waiting for enable.
        IDLE,
        // Counting through the PWM period.
        COUNTING,
        // Period complete, pulse period_done for one cycle.
        DONE
    } t_state;

    t_state state, next_state;

    // Period counter register.
    logic [WIDTH-1:0] count;

    // p_state_reg: Clocked state and counter register.
    //
    // Captures next_state on each rising edge of clk.
    // Synchronous reset drives state to IDLE and clears counter.
    // Counter increments each cycle in COUNTING state.
    //
    // // wavedrom::
```

(continues on next page)

(continued from previous page)

```
    //
    //    { "signal": [
    //      { "name": "clk",   "wave": "P....." },
    //      { "name": "rst",   "wave": "1.0....." },
    //      { "name": "en",    "wave": "0....." },
    //      { "name": "state", "wave": "=.=.....", "data": ["(any)","IDLE"] },
    //      { "name": "pwm_o", "wave": "x.0....." }
    //    ]}
    always_ff @(posedge clk) begin : p_state_reg
        if (rst) begin
            state <= IDLE;
            count <= '0;
        end else begin
            state <= next_state;
            if (state == COUNTING)
                count <= count + 1'b1;
            else
                count <= '0;
        end
    end

    // p_next_state: Combinational next-state logic.
    //
    // FSM advances only when en is asserted.
    // COUNTING state runs for a full 256-cycle period then moves to DONE.
    //
    // Transitions:
    //   IDLE     + en           -> COUNTING
    //   COUNTING + count==255   -> DONE
    //   DONE                    -> IDLE
    //
    // // wavedrom::
    //
    //    { "signal": [
    //      { "name": "clk",         "wave": "P............." },
    //      { "name": "en",          "wave": "01..........0" },
    //      { "name": "state",       "wave": "=.=........=..", "data": ["IDLE","COUNTING
↪","DONE"] },
    //      { "name": "count",       "wave": "=.=........=..", "data": ["0","1..255","0
↪"] },
    //      { "name": "period_done", "wave": "0.........1.0" }
    //    ]}
    always_comb begin : p_next_state
        next_state = state;
        case (state)
            IDLE:     if (en)              next_state = COUNTING;
            COUNTING: if (count == 8'hFF)  next_state = DONE;
            DONE:                          next_state = IDLE;
            default:                       next_state = IDLE;
        endcase
    end

```

(continues on next page)

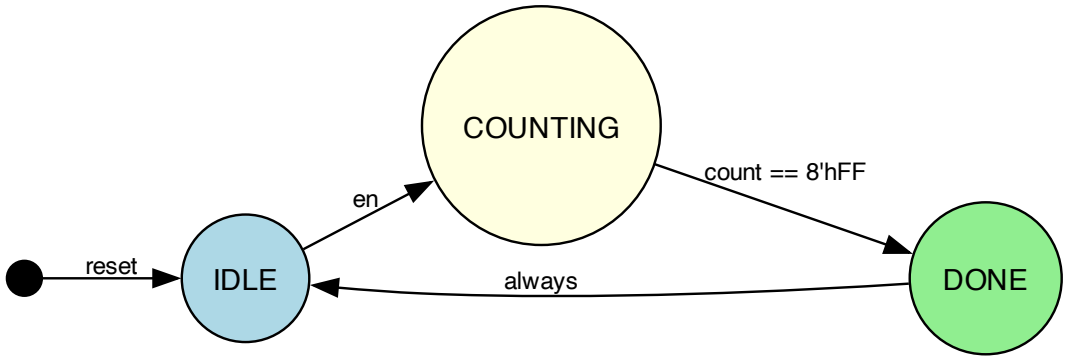
(continued from previous page)

```
    // p_outputs: Combinational Moore output decode.
    //
    // PWM output is high when counter is below duty_i threshold.
    // period_done pulses for exactly one cycle when state is DONE.
    //
    // Output truth table:
    //   IDLE     -> pwm_o=0 period_done=0
    //   COUNTING -> pwm_o=(count < duty_i) period_done=0
    //   DONE     -> pwm_o=0 period_done=1
    //
    // // wavedrom::
    //
    //    { "signal": [
    //      { "name": "state",       "wave": "=.=.=.",     "data": ["IDLE","COUNTING",
↪"DONE"] },
    //      { "name": "count",       "wave": "=.=.=.",     "data": ["x","0..N","N"] },
    //      { "name": "duty_i",      "wave": "=.....",     "data": ["D"] },
    //      { "name": "pwm_o",       "wave": "0.x.0." },
    //      { "name": "period_done", "wave": "0...01" }
    //    ]}
    always_comb begin : p_outputs
        pwm_o       = 1'b0;
        period_done = 1'b0;
        case (state)
            IDLE:     begin pwm_o = 1'b0;           period_done = 1'b0; end
            COUNTING: begin pwm_o = (count < duty_i); period_done = 1'b0; end
            DONE:     begin pwm_o = 1'b0;           period_done = 1'b1; end
            default:  begin pwm_o = 1'b0;           period_done = 1'b0; end
        endcase
    end

endmodule
```

2.6.3.2 pwm_controller State Machine

Auto-extracted from `pwm_controller.sv`.

State Transition Diagram



State Output Table

State	pwm_o	period_done
IDLE	0	0
COUNTING	0	0
DONE	0	1

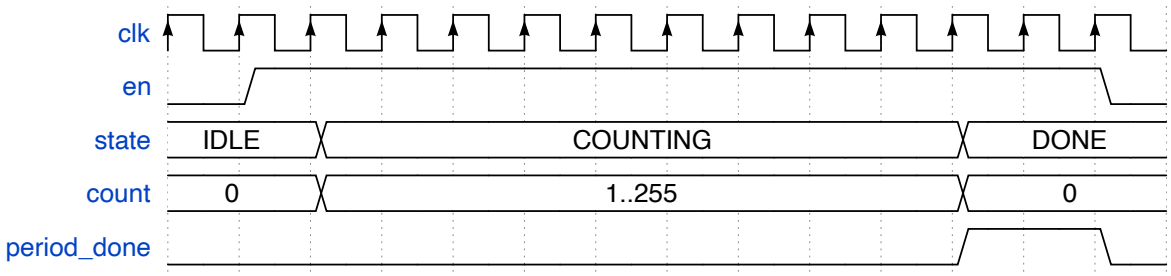
Transitions

From	To	Condition
IDLE	COUNTING	en
COUNTING	DONE	count == 8'hFF
DONE	IDLE	(always)

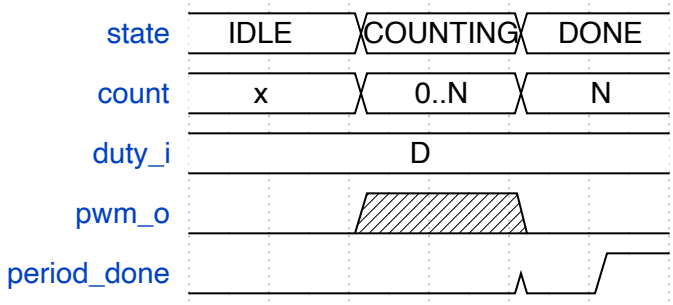
2.6.3.3 pwm_controller — Timing Diagrams

All timing diagrams extracted from `pwm_controller.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

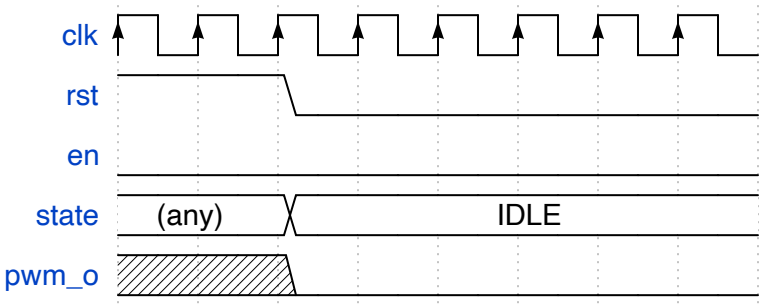
P Next State



P Outputs



P State Reg



2.6.3.4 Processes

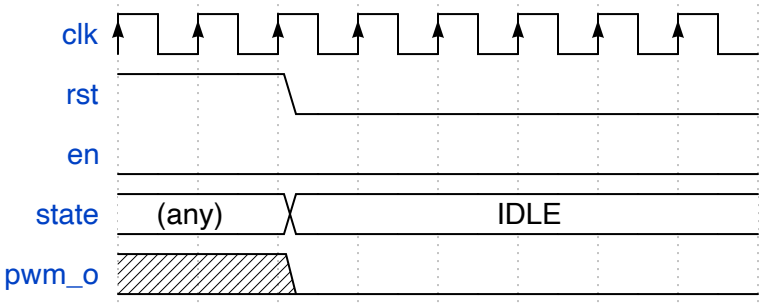
Auto-extracted from `pwm_controller.sv`.

p_state_reg

Source file: `pwm_controller.sv`
Source line: 53
Block type: `always_ff`
Sensitivity list: `posedge clk`

Description

p_state_reg: Clocked state and counter register. Captures next_state on each rising edge of clk. Synchronous reset drives state to IDLE and clears counter. Counter increments each cycle in COUNTING state.



Source

```
always_ff @(posedge clk) begin : p_state_reg
    if (rst) begin
        state <= IDLE;
        count <= '0;
    end else begin
        state <= next_state;
        if (state == COUNTING)
            count <= count + 1'b1;
        else
            count <= '0;
    end
end
```

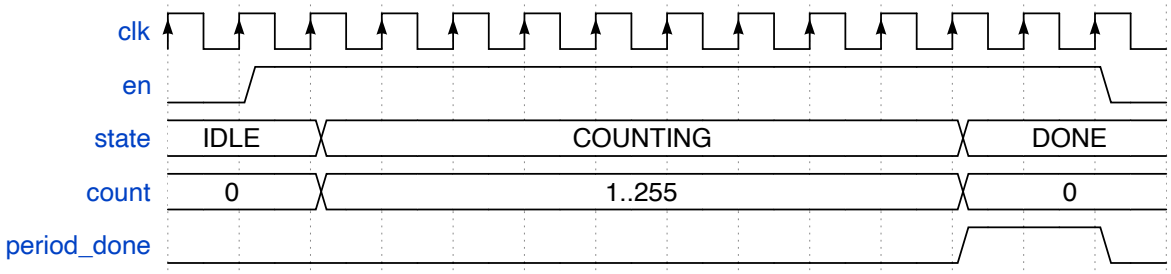
p_next_state

Source file: `pwm_controller.sv`
Source line: 85
Block type: `always_comb`
Sensitivity list: *

Description

p_next_state: Combinational next-state logic. FSM advances only when en is asserted. COUNTING state runs for a full 256-cycle period then moves to DONE. Transitions: .. code-block:: none

IDLE + en -> COUNTING COUNTING + count==255 -> DONE DONE -> IDLE



Source

```
always_comb begin : p_next_state
    next_state = state;
    case (state)
        IDLE:     if (en)              next_state = COUNTING;
        COUNTING: if (count == 8'hFF)  next_state = DONE;
        DONE:                          next_state = IDLE;
        default:                       next_state = IDLE;
    endcase
end
```

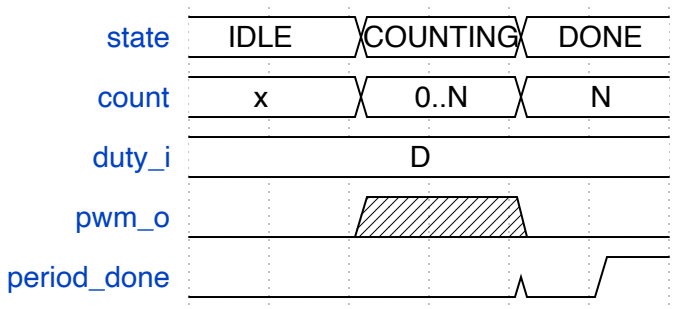
p_outputs

Source file: `pwm_controller.sv`
Source line: 114
Block type: `always_comb`
Sensitivity list: *

Description

p_outputs: Combinational Moore output decode. PWM output is high when counter is below duty_i threshold. period_done pulses for exactly one cycle when state is DONE. Output truth table: .. code-block:: none

> IDLE -> pwm_o=0 period_done=0 COUNTING -> pwm_o=(count < duty_i) period_done=0 DONE -> pwm_o=0 period_done=1



Source

```
always_comb begin : p_outputs
    pwm_o       = 1'b0;
    period_done = 1'b0;
    case (state)
        IDLE:     begin pwm_o = 1'b0;           period_done = 1'b0; end
        COUNTING: begin pwm_o = (count < duty_i); period_done = 1'b0; end
        DONE:     begin pwm_o = 1'b0;           period_done = 1'b1; end
        default:  begin pwm_o = 1'b0;           period_done = 1'b0; end
    endcase
end
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_state_reg	`always_ff`	`posedge clk`	53	✓
p_next_state	`always_comb`	*	85	✓
p_outputs	`always_comb`	*	114	✓

2.6.3.5 pwm_controller — Clock Domain Analysis

Auto-extracted from `pwm_controller.sv`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
`clk`	`p_state_reg`

2.6.4 traffic_light

2.6.4.1 traffic_light — Entity

Source file: `src/traffic_light.vhd`.

ENTITY **`traffic_light`** IS

Traffic light controller.

Implements a standard UK traffic light sequencer with four states: RED, RED_AMBER, GREEN, and AMBER. The FSM advances on each rising clock edge when timer_exp is asserted.

State sequence: RED -> RED_AMBER -> GREEN -> AMBER -> RED. **`Generics`**

Generic	Type	Default	Description

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	System clock, rising-edge triggered.
rst	std_logic	in	Synchronous active-high reset. Returns FSM to RED state.
timer_exp	std_logic	in	Phase timer expiry pulse. Must be high for exactly one cycle.
red_out	std_logic	out	Red lamp drive. Asserted in RED and RED_AMBER states.
amber_out	std_logic	out	Amber lamp drive. Asserted in RED_AMBER and AMBER states.
green_out	std_logic	out	Green lamp drive. Asserted in GREEN state only.

Annotated Source

Listing 5: src/traffic_light.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Traffic light controller.
--
-- Implements a standard UK traffic light sequencer with four states:
-- RED, RED_AMBER, GREEN, and AMBER. The FSM advances on each rising
-- clock edge when timer_exp is asserted.
--
```

(continues on next page)

(continued from previous page)

```
-- State sequence: RED -> RED_AMBER -> GREEN -> AMBER -> RED.
entity traffic_light is
    port (
        -- System clock, rising-edge triggered.
        clk       : in  std_logic;
        -- Synchronous active-high reset. Returns FSM to RED state.
        rst       : in  std_logic;
        -- Phase timer expiry pulse. Must be high for exactly one cycle.
        timer_exp : in  std_logic;
        -- Red lamp drive. Asserted in RED and RED_AMBER states.
        red_out   : out std_logic;
        -- Amber lamp drive. Asserted in RED_AMBER and AMBER states.
        amber_out : out std_logic;
        -- Green lamp drive. Asserted in GREEN state only.
        green_out : out std_logic
    );
end entity traffic_light;

-- RTL implementation using a 3-process Moore FSM style.
architecture rtl of traffic_light is

    -- FSM state type encoding all four traffic light phases.
    type t_state is (
        -- Red light only. Vehicles must remain stopped.
        RED,
        -- Red and amber simultaneously. Vehicles prepare to move.
        RED_AMBER,
        -- Green light. Vehicles may proceed.
        GREEN,
        -- Amber light only. Vehicles prepare to stop.
        AMBER
    );

    -- Current state register, initialised to RED at power-on.
    signal state      : t_state := RED;
    -- Next state combinational signal, feeds state register.
    signal next_state : t_state;

begin

    -- p_state_reg: Clocked state register.
    --
    -- Captures next_state on each rising edge of clk.
    -- When rst is asserted the state returns to RED synchronously,
    -- regardless of timer_exp or next_state.
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",   "wave": "P....." },
    --      { "name": "rst",   "wave": "1.0....." },
    --      { "name": "state", "wave": "=.=.....", "data": ["(any)","RED"] },
```

(continues on next page)

(continued from previous page)

```
    --      { "name": "red",   "wave": "x.1....." }
    --    ]}
    p_state_reg : process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state <= RED;
            else
                state <= next_state;
            end if;
        end if;
    end process p_state_reg;

    -- p_next_state: Combinational next-state logic.
    --
    -- Evaluates the FSM transition function. Default assignment
    -- next_state <= state implements hold when timer_exp is low.
    -- All four transitions are guarded solely by timer_exp = 1.
    --
    -- Transitions:
    --   RED       + timer_exp -> RED_AMBER
    --   RED_AMBER + timer_exp -> GREEN
    --   GREEN     + timer_exp -> AMBER
    --   AMBER     + timer_exp -> RED
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",       "wave": "P........." },
    --      { "name": "timer_exp", "wave": "0.1.1.1.1." },
    --      { "name": "state",     "wave": "=.=.=.=.=.",
    --        "data": ["RED","RED_AMB","GREEN","AMBER","RED"] },
    --      { "name": "red_out",   "wave": "1.1.0.0.1." },
    --      { "name": "amber_out", "wave": "0.1.0.1.0." },
    --      { "name": "green_out", "wave": "0.0.1.0.0." }
    --    ]}
    p_next_state : process(state, timer_exp)
    begin
        next_state <= state;

        case state is
            when RED =>
                if timer_exp = '1' then
                    next_state <= RED_AMBER;
                end if;

            when RED_AMBER =>
                if timer_exp = '1' then
                    next_state <= GREEN;
                end if;

            when GREEN =>
```

(continues on next page)

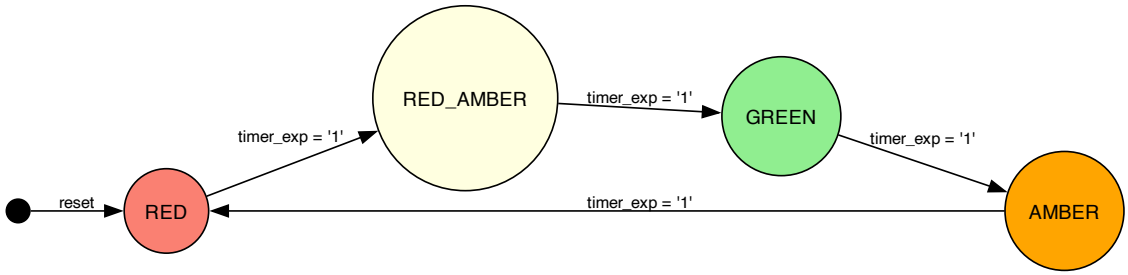
(continued from previous page)

```
                if timer_exp = '1' then
                    next_state <= AMBER;
                end if;

            when AMBER =>
                if timer_exp = '1' then
                    next_state <= RED;
                end if;
        end case;
    end process p_next_state;

    -- p_outputs: Combinational Moore output decode.
    --
    -- Drives lamp outputs based solely on current state.
    -- All outputs default to 0 before the case statement to
    -- prevent latches during synthesis.
    --
    -- Output truth table:
    --   RED       -> red=1 amber=0 green=0
    --   RED_AMBER -> red=1 amber=1 green=0
    --   GREEN     -> red=0 amber=0 green=1
    --   AMBER     -> red=0 amber=1 green=0
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "state",     "wave": "=.=.=.=.",
    --        "data": ["RED","RED_AMB","GREEN","AMBER"] },
    --      { "name": "red_out",   "wave": "1.1.0.0." },
    --      { "name": "amber_out", "wave": "0.1.0.1." },
    --      { "name": "green_out", "wave": "0.0.1.0." }
    --    ]}
    p_outputs : process(state)
    begin
        red_out   <= '0';
        amber_out <= '0';
        green_out <= '0';

        case state is
            -- define red
            when RED       => red_out <= '1';
            when RED_AMBER => red_out <= '1'; amber_out <= '1';
            when GREEN     => green_out <= '1';
            when AMBER     => amber_out <= '1';
        end case;
    end process p_outputs;

end architecture rtl;
```

2.6.4.2 traffic_light State Machine

Auto-extracted from `traffic_light.vhd`.

State Transition Diagram



State Output Table

State	red_out	amber_out	green_out
RED	1	0	0
RED_AMBER	1	1	0
GREEN	0	0	1
AMBER	0	1	0

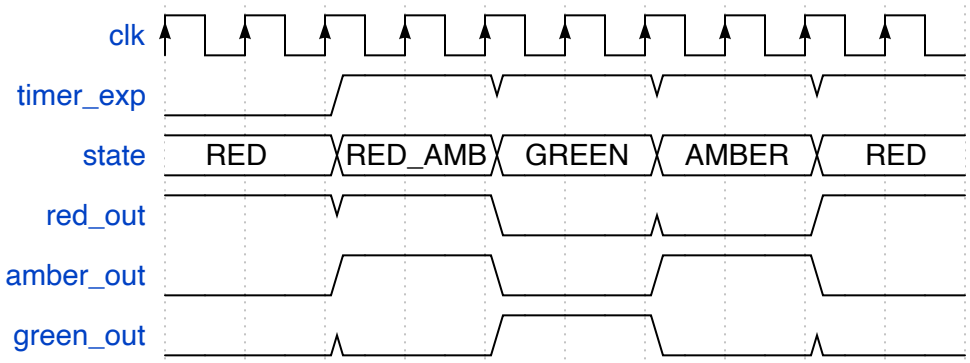
Transitions

From	To	Condition
RED	RED_AMBER	`timer_exp = '1'`
RED_AMBER	GREEN	`timer_exp = '1'`
GREEN	AMBER	`timer_exp = '1'`
AMBER	RED	`timer_exp = '1'`

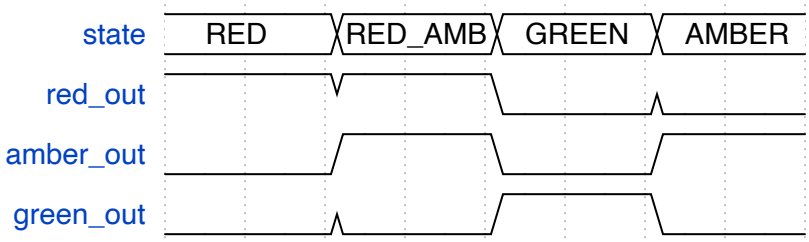
2.6.4.3 traffic_light — Timing Diagrams

All timing diagrams extracted from `traffic_light.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

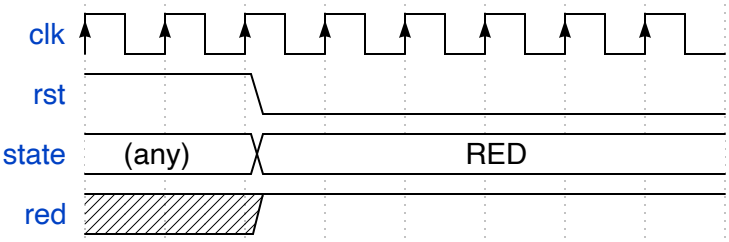
P Next State



P Outputs



P State Reg



2.6.4.4 Processes

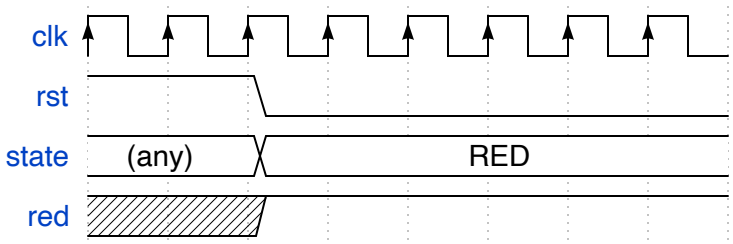
Auto-extracted from `traffic_light.vhd`.

p_state_reg

Source file: `traffic_light.vhd`
Source line: 65
Block type: `process`
Sensitivity list: `clk`

Description

p_state_reg: Clocked state register. Captures next_state on each rising edge of clk. When rst is asserted the state returns to RED synchronously, regardless of timer_exp or next_state.



Source

```
p_state_reg : process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            state <= RED;
        else
            state <= next_state;
        end if;
    end if;
end process p_state_reg;
```

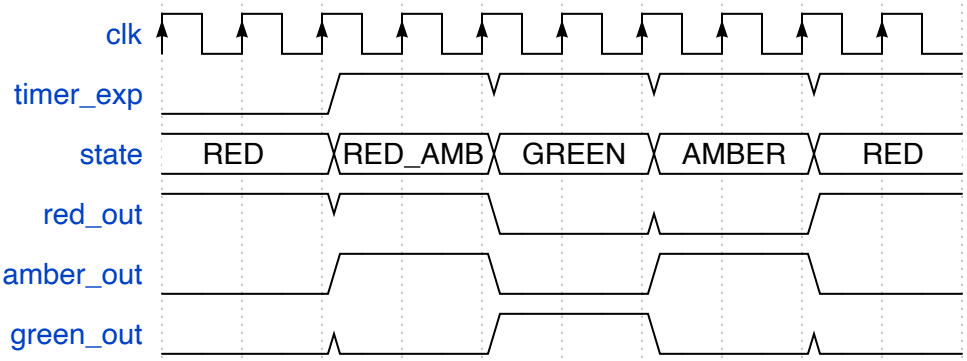
p_next_state

Source file: `traffic_light.vhd`
Source line: 99
Block type: `process`
Sensitivity list: `state`, `timer_exp`

Description

p_next_state: Combinational next-state logic. Evaluates the FSM transition function. Default assignment next_state <= state implements hold when timer_exp is low. All four transitions are guarded solely by timer_exp = 1. Transitions: .. code-block:: none

RED + timer_exp -> RED_AMBER RED_AMBER + timer_exp -> GREEN GREEN + timer_exp -> AMBER AMBER + timer_exp -> RED



Signal Assignments

State	next_state
RED	RED_AMBER
RED_AMBER	GREEN
GREEN	AMBER
AMBER	RED

Source

```
p_next_state : process(state, timer_exp)
begin
    next_state <= state;

    case state is
        when RED =>
            if timer_exp = '1' then
                next_state <= RED_AMBER;
            end if;

        when RED_AMBER =>
            if timer_exp = '1' then
                next_state <= GREEN;
            end if;

        when GREEN =>
            if timer_exp = '1' then
                next_state <= AMBER;
            end if;

        when AMBER =>
            if timer_exp = '1' then
                next_state <= RED;
            end if;
    end case;
end process p_next_state;
```

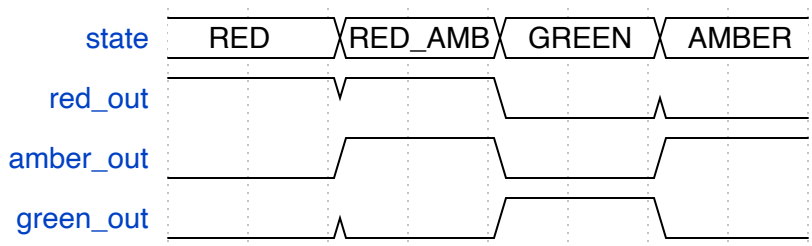
p_outputs

Source file: `traffic_light.vhd`
Source line: 147
Block type: `process`
Sensitivity list: `state`

Description

p_outputs: Combinational Moore output decode. Drives lamp outputs based solely on current state. All outputs default to 0 before the case statement to prevent latches during synthesis. Output truth table: .. code-block:: none

> RED -> red=1 amber=0 green=0 RED_AMBER -> red=1 amber=1 green=0 GREEN -> red=0 amber=0 green=1 AMBER -> red=0 amber=1 green=0



Source

```
p_outputs : process(state)
begin
    red_out   <= '0';
    amber_out <= '0';
    green_out <= '0';

    case state is
        -- define red
        when RED       => red_out <= '1';
        when RED_AMBER => red_out <= '1'; amber_out <= '1';
        when GREEN     => green_out <= '1';
        when AMBER     => amber_out <= '1';
    end case;
end process p_outputs;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_state_reg	process	clk	65	✓
p_next_state	process	state, timer_exp	99	✓
p_outputs	process	state	147	✓

2.6.4.5 traffic_light — Clock Domain Analysis

Auto-extracted from `traffic_light.vhd`.

> **Note**
>
> No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

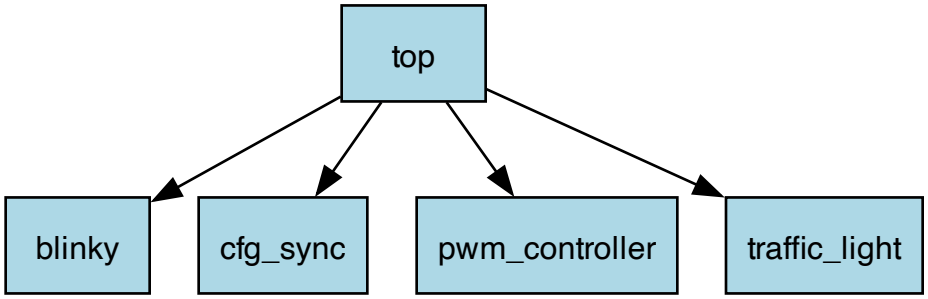
Clock	Clocked Processes
clk	p_state_reg

Chapter 3

Design Hierarchy

Top-level module: `top` Total modules: 5

3.1 Instantiation Tree



3.2 Module List

Module	Source File	Parents	Children
blinky	`src/blinky.vhd`	`top`	—
cfg_sync	`src/cfg_sync.vhd`	`top`	—
pwm_controller	`src/pwm_controller.sv`	`top`	—
top	`src/top.vhd`	*(top)*	`blinky`, `cfg_sync`, `pwm_controller`, `traffic_light`
traffic_light	`src/traffic_light.vhd`	`top`	—

Chapter 4

Indices

- genindex
- search